



**UNIVERSIDAD AUTÓNOMA DEL
ESTADO DE MÉXICO
FACULTAD DE INGENIERÍA**



Licenciatura

Ingeniería en computación.

Unidad de aprendizaje:

Proyecto integral de comunicación de datos.

Grupo: 01.

Profesor:

Juan Carlos Escobar González.

Entregable 21: Plan de Pruebas.

Plataforma inteligente de monitoreo de redes.

Equipo de trabajo:

Daniel Dávila Lara.

Jesús Omar Carranza Colín.

Fecha de entrega

07/mayo/2026

1. Alcance del proyecto

El entregable seleccionado es el sistema de monitoreo y detección de anomalías en tráfico IoT mediante dashboard web, que comprende los módulos de captura de paquetes, filtrado y almacenamiento para la clasificación de tráfico (normal/anómalo), generación de alertas y visualización en el dashboard web sobre una red IoT con dispositivos ESP32.

Alcance del plan.

- Entorno: Red LAN privada en el laboratorio de redes de la Facultad de ingeniería UAEM.
- El sistema debe capturar tráfico de red generado por dispositivos IoT simulados.
- El sistema debe identificar tráfico anómalo (alto volumen de paquetes o comportamiento sospechoso).
- El dashboard web debe mostrar alertas y métricas en tiempo casi real.
- El sistema debe registrar eventos detectados para análisis posterior.
- El sistema debe funcionar correctamente dentro de una red local Linux de laboratorio.

Requisitos técnicos.

- Captura de paquetes en tiempo real con Scapy.
- Filtrado por protocolo de comunicación.
- Extracción de 5 atributos por paquete: IP origen, IP destino, puerto origen, puerto destino, protocolo.
- Clasificación de tráfico como 'normal' o 'anómalo' con precisión $\geq 85\%$.
- Generación de alerta al detectar anomalía, visible en logs o dashboard.
- Procesamiento y clasificación de cada paquete en ≤ 500 ms.

2. Casos de Prueba

Caso de Prueba 1 – Captura de tráfico IoT

Campo	Descripción
Nombre	Captura de tráfico de red IoT
Propósito	Validar la captura de tráfico generado por dispositivos IoT simulados.
Precondiciones	Red local configurada, ESP32 conectados, sistema Linux activo y script de captura ejecutándose.
Pasos	1. Encender dispositivos IoT. 2. Generar tráfico de red. 3. Ejecutar captura con Python/Scapy. 4. Verificar recepción de paquetes.
Datos de entrada	Paquetes y tráfico TCP generados por los dispositivos IoT.
Resultado esperado	El sistema captura correctamente los paquetes y los registra.
Resultado obtenido	Captura exitosa de paquetes provenientes de dispositivos IoT simulados.
Incidencias	No se detectaron incidencias.

Caso de Prueba 2 – Detección de anomalías

Campo	Descripción
Nombre	Detección de tráfico anómalo
Propósito	Validar que el sistema detecte comportamientos anómalos en la red.
Precondiciones	Sistema de monitoreo activo y herramienta hping3 disponible.
Pasos	1. Ejecutar tráfico normal. 2. Generar tráfico masivo usando hping3. 3. Observar comportamiento del sistema. 4. Verificar generación de alertas.
Datos de entrada	Incremento artificial de paquetes TCP/ICMP.
Resultado esperado	El sistema detecta el tráfico anómalo y genera una alerta.
Resultado obtenido	Se detectó incremento anormal de tráfico y se registró la anomalía.
Incidencias	Ligero retraso para la visualización del dashboard.

Caso de Prueba 3 – Funcionamiento del dashboard web

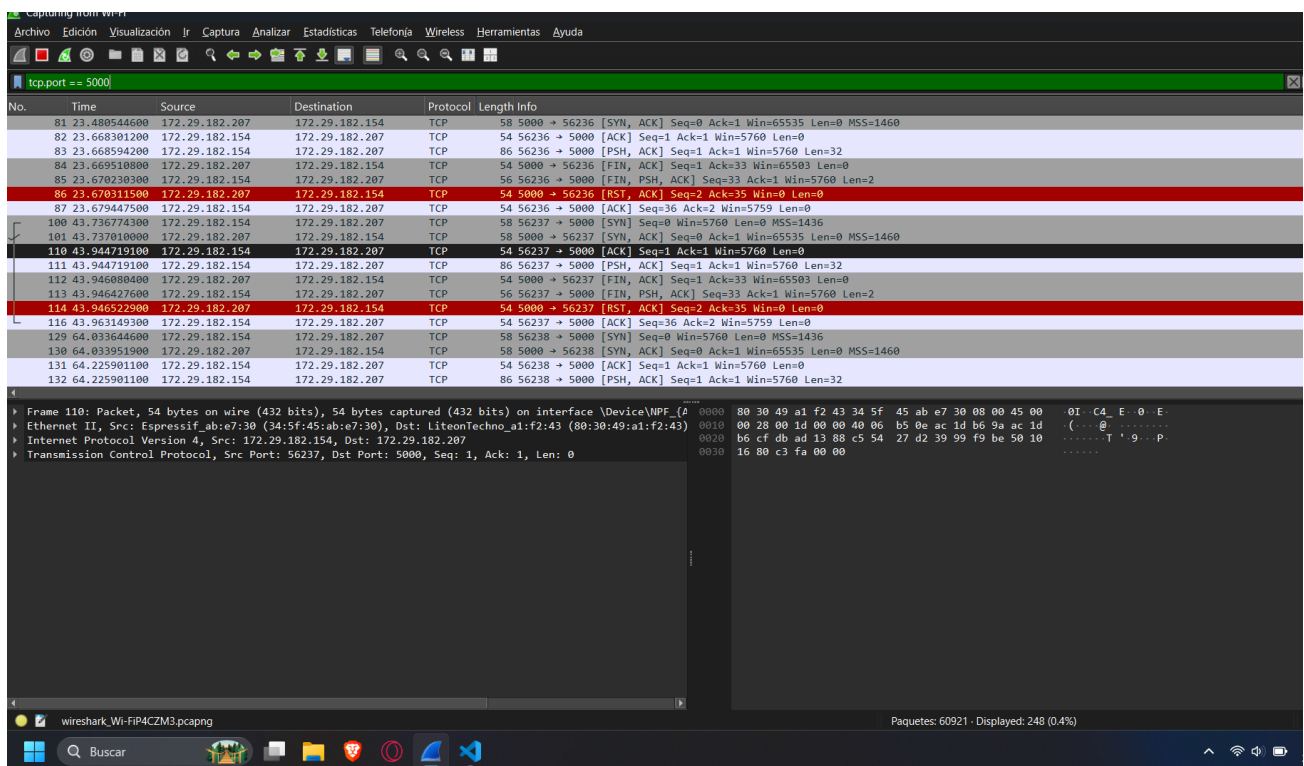
Campo	Descripción
Nombre	Validación del dashboard web
Propósito	Verificar que el dashboard muestre métricas y alertas correctamente.
Precondiciones	Backend activo, base de datos funcionando y navegador web disponible.
Pasos	1. Iniciar dashboard web. 2. Generar tráfico en la red. 3. Revisar actualización de métricas. 4. Verificar visualización de alertas.
Datos de entrada	Eventos de red generados por dispositivos IoT.
Resultado esperado	El dashboard muestra gráficas, métricas y alertas correctamente.
Resultado obtenido	Visualización correcta de estadísticas y alertas de red.
Incidencias	Fallo de conectividad con los dos módulos anteriores

3. Criterios de aceptación

ID	Criterio de aceptación	Método de validación
RF-01	El sistema debe capturar al menos el 95% de los paquetes enviados desde los dispositivos IoT en un tiempo menor a 1 segundo.	Verificación con Wireshark y revisión en los logs.
RF-02	Solo deben registrarse paquetes de los protocolos permitidos (TCP, UDP, ICMP y HTTP).	Revisión de logs y validación del tráfico capturado en Wireshark.
RF-03	El sistema debe mostrar correctamente los datos principales de los paquetes (IP origen, IP destino, puertos y protocolo) en al menos el 95% de las pruebas realizadas.	Validación mediante script en Python y revisión manual de registros.
RF-04	El sistema de detección debe identificar correctamente al menos el 80% de los eventos normales y anómalos.	Evaluación mediante pruebas controladas.
RF-05	Cada anomalía generada debe producir una alerta visible en el dashboard en menos de 2 segundos.	Revisión de los log y validación visual en el dashboard web.

4. Herramientas y registro de evidencias

Herramienta	Uso en el proyecto	Evidencia generada
Python + Scapy	Captura y análisis de paquetes de red en tiempo real.	Logs de captura, registros de paquetes y capturas de pantalla de ejecución.
hping3	Generación de tráfico normal y tráfico anómalo para pruebas.	Capturas de terminal con tráfico generado y resultados de prueba.
Wireshark	Monitoreo y validación visual del tráfico de red capturado.	Archivos .pcap y capturas de pantalla del análisis de tráfico.
Dashboard Web (Flask)	Visualización de métricas, eventos y alertas de anomalías.	Capturas de pantalla del dashboard mostrando gráficas y alertas.
Scripts en Python	Validación de datos capturados y detección de anomalías.	Reportes generados y registros de resultados en logs.
Ping	Verificación de conectividad entre dispositivos y red local.	Resultados mostrados en terminal y capturas de pantalla.



```
File Edit Selection View Go Run Terminal Help
server.py X scrapy.py
C:\Users\Lenovo\Downloads> server.py > ...
1
2
3 HOST = "0.0.0.0"
4 PORT = 5000
5
6 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7 server.bind((HOST, PORT))
8 server.listen()
9
10 print("Servidor escuchando...")
11
12 while True:
13     conn, addr = server.accept()
14     print("\nConectado por:", addr)
15
16     data = conn.recv(1024)
17     if data:
18         print("Mensaje recibido:", data.decode())
19
20     conn.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Lenovo/Downloads/server.py

Conectado por: ('172.29.182.154', 62564)
Mensaje recibido: Usuario=user 1 | Password=I9J9WZ

Conectado por: ('172.29.182.154', 62565)
Mensaje recibido: Usuario=user 2 | Password=UV5F8Z

Conectado por: ('172.29.182.154', 62566)
Mensaje recibido: Usuario=user 3 | Password=XC54NQ

```
File Edit Selection View Go Run Terminal Help
EXPLORER
NO FOLDER OPENED
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
You can clone a repository locally.
Clone Repository
To learn more about how to use Git and source control in VS Code read our docs.
server.py X scrapy.py
C:\Users\Lenovo\Downloads> scrapy.py > capturar
1 from datetime import datetime
2
3
4
5 logs = []
6
7
8 def capturar(packet):
9     if packet.haslayer(IP) and packet.haslayer(TCP):
10
11
12         ip_src = packet[IP].src
13         ip_dst = packet[IP].dst
14         sport = packet[TCP].sport
15         dport = packet[TCP].dport
16
17
18         if dport == 5000 or sport == 5000:
19
20
21             if packet.haslayer(Raw):
22
23                 contenido = packet[Raw].load.decode(errors="ignore").strip()
24
25
26                 if contenido != "":
27
28                     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
29
30
31                     log = f"""
32                     [{timestamp}]
33                     origen: {ip_src}:{sport}
34
35 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Lenovo>
PS C:\Users\Lenovo>
```

Ln 11, Col 1 Spaces: 4 UTF-8 CRLF Python 3.13.7 Python 3.13 (64-bit) Go Live Miku

5. Integración al proyecto

5.1 Controlar la calidad

Los resultados de las pruebas se compararán con los criterios de aceptación definidos previamente para verificar que el sistema funcione correctamente.

Durante las pruebas se evaluará:

- La captura correcta de tráfico de red.
- La detección de anomalías.
- El funcionamiento del dashboard web.
- El registro de eventos y alertas.

Si alguna prueba no cumple con lo esperado, se revisará el sistema para identificar el problema y realizar ajustes necesarios, como modificar parámetros de detección, corregir errores de código o mejorar la visualización del dashboard.

5.2 Validar el alcance

El entregable será considerado aceptado cuando:

- El sistema capture tráfico correctamente.
- Las anomalías sean detectadas y registradas.
- El dashboard muestre métricas y alertas funcionales.
- Existan evidencias de pruebas realizadas (capturas, logs y archivos de red).
- El asesor valide el funcionamiento general del proyecto.

5.3 Mejoras propuestas

En caso de detectar problemas durante las pruebas, se plantean las siguientes mejoras:

- Ajustar parámetros para mejorar la detección de anomalías.
- Optimizar el procesamiento de paquetes para mejorar el rendimiento.
- Mejorar la actualización y estabilidad del dashboard web.
- Incorporar nuevas métricas o visualizaciones para futuras versiones del sistema.